```
@Ignore                                          <-- This test will now be skipped.
@Test
public void should_obtain_a_new_status_when_enough_points_are_earned() {
    ...
}
```

You could then implement the `StatusService` class or interface, and the `statusLevelFor()` method, using a test-driven approach (an example of doing this is illustrated in section 10.3.7). Once the method works, you'd come back and remove the `@Ignore` annotation, to ensure that the method works as expected in its original context.

This approach is simple and intuitive. But it does mean that you'll momentarily have more than one unit test in progress, and you'll need to remember to go back and restore the skipped test later on.

10.3.5 Use a minimal implementation

If the service class looks too complicated to implement in one go, another option is to code a minimal implementation of the real service class, and to use that for your test. This can help you get a feel for an API and how the classes should interact. Once you've finished with the `FrequentFlyer` code, for example, you'd come back and flesh out the status service implementation. Technically, this is an integration test rather than a unit test, which may have performance implications down the track.

10.3.6 Use stubs and mocks to defer the implementation of more complex code

Another approach, commonly used in outside-in development, is to use a stub or mock class to act as a placeholder for the real implementation, and at the same time to describe the behavior you expect of the new class. A stub or mock class is essentially a class you can control for testing purposes that stands in for a real class.

Using stubs and mocks helps you focus on the test at hand and avoids having several unit tests in progress at the same time. It also helps you clarify the roles and responsibilities of the classes and services in your application, which is very useful for larger, more complex applications. But it does add some overhead and complexity to your test classes.

> **USING MOCKS AND STUBS** For a more detailed discussion of using mocks and stubs to discover and define application design, be sure to read the seminal book *Growing Object-Oriented Software, Guided by Tests*, by Steve Freeman and Nat Pryce (Addison Wesley Professional, 2009).

Let's see what this approach would look like for the `StatusService` classes.

MAKE SURE YOU CAN PROVIDE A CUSTOM VERSION OF THE SERVICE TO YOUR CLASS

Remember, you need the `StatusService` class to tell you what status a Frequent Flyer member should achieve with a given number of points: